

Socket

Lezione 15

Sistemi Operativi - II Modulo (Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



SAPIENZA
UNIVERSITÀ DI ROMA

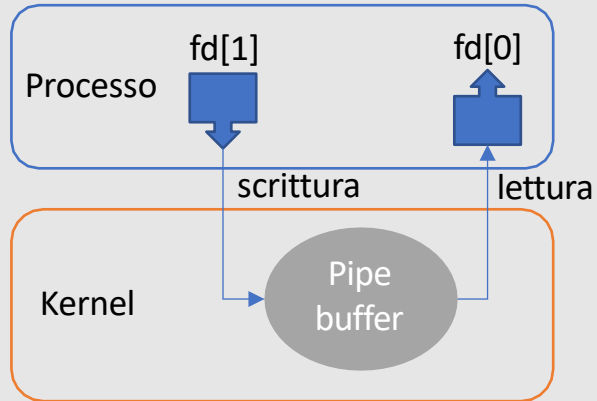


Agenda

- Pipe – riepilogo
- Socket

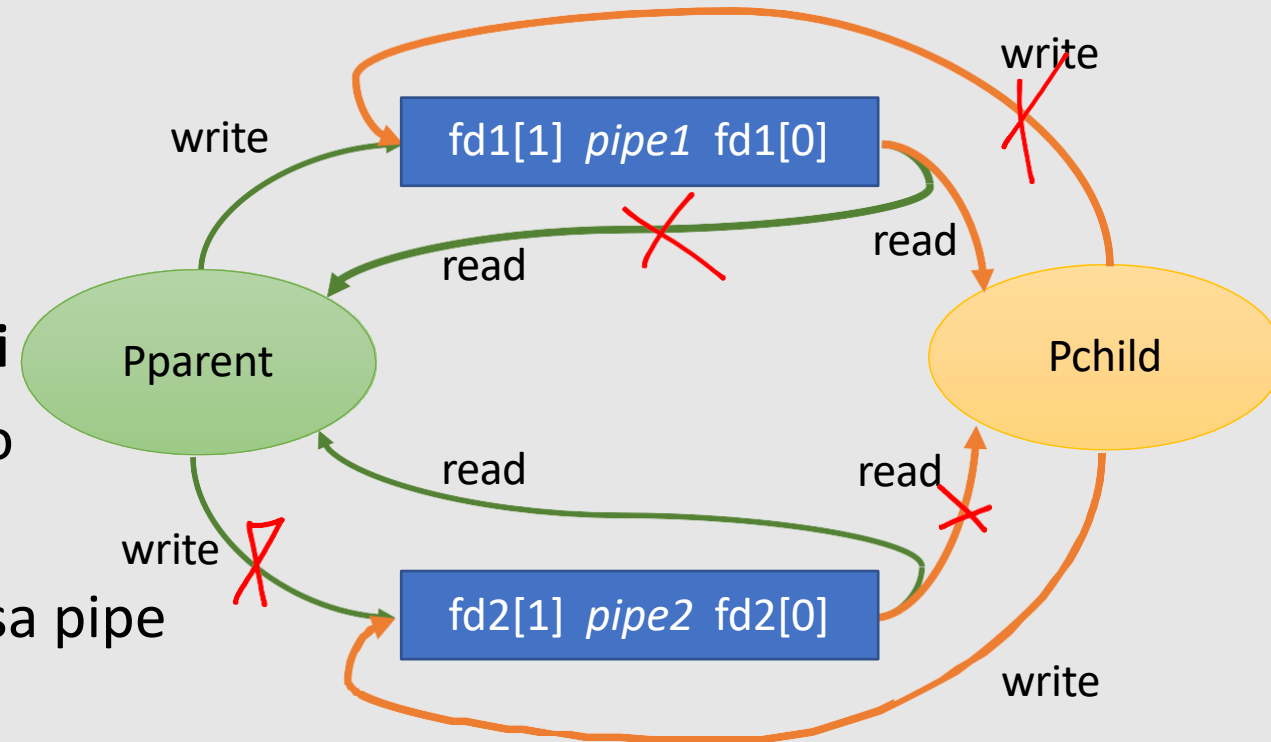


Pipe - riepilogo



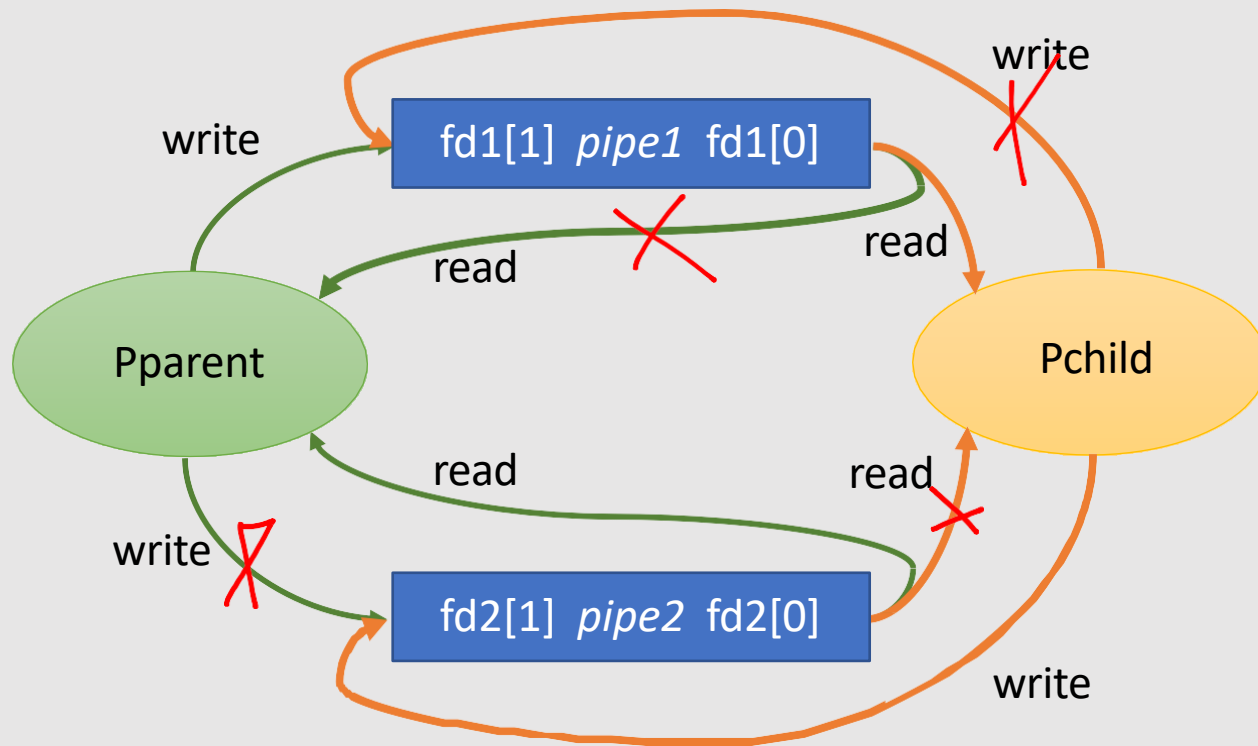
- Pipe: usate per IPC, sono **unidirezionali**
- Dopo un fork, Pparent e Pchild possono leggere e scrivere
- Se un processo scrive e legge dalla stessa pipe ottiene quello che ha scritto
- Quindi, per comunicare *bidirezionalmente*:
 - una pipe per Pparent -> Pchild e
 - una pipe per Pchild -> Pparent

Stato delle connessioni alle *pipe1* e *pipe2* dopo la fork



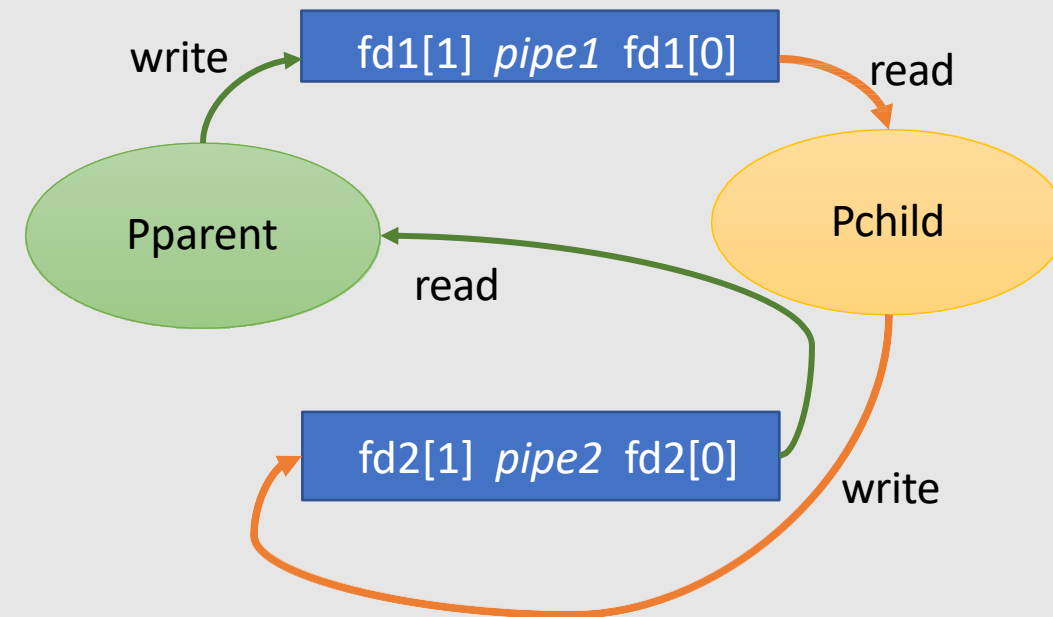
Pipe - riepilogo

Stato delle connessioni alle *pipe1* e *pipe2* dopo la fork



Stato delle connessioni alle *pipe1* e *pipe2* dopo la chiusura appropriata dei canali di R e W

- Pchild: `close(fd1[1]); close(fd2[0])`
- Pparent: `close(fd1[0]); close(fd2[1])`



Pipe - riepilogo

- Vediamo `pipe-esempio.c`



Socket



Socket

- Consentono la comunicazione tra processi (anche residenti su macchine diverse) nel paradigma **client-server**
- **Server:**
 - Definisce la socket (“presa”)
 - Il riferimento (nome di file o indirizzo di rete) è noto al client
 - Accetta connessioni sulla socket da parte di uno o più client: ricava un file descriptor (full-duplex) sulla connessione
- **Client:**
 - Definisce la socket
 - Crea una connessione sulla socket: ricava un file descriptor (full-duplex) sulla connessione

System call di interesse

- `socket`
 - crea struttura dati della socket
- `bind`
 - associa un nome alla socket
- `listen`
 - mette un processo in ascolto su di una socket
- `accept`
 - accetta una connessione su di una socket

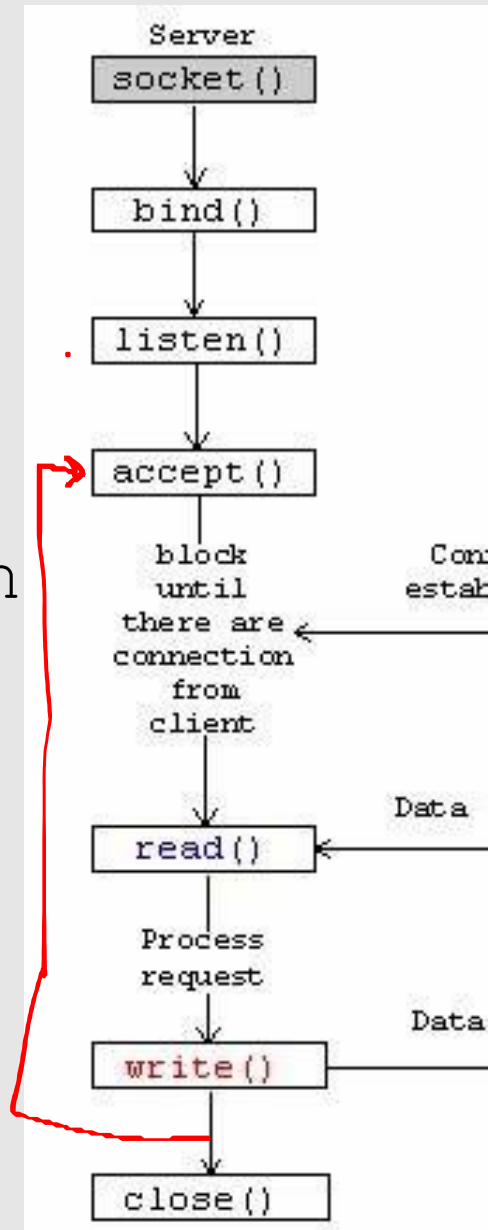
Tipologia Socket: definite da 3 attributi

- Domain (o family) - modalità collegamento
 - AF_LOCAL (o AF_UNIX): client e server devono risiedere sulla stessa macchina
 - AF_INET: client e server comunicano in rete con protocollo IPV4
 - AF_INET6: client e server comunicano in rete con protocollo IPV6
- Type - semantica del collegamento
 - SOCK_STREAM: flusso bidirezionale di byte affidabile basato su connessione (socket TCP). Supporta notifiche asincrone (Out of Band)
 - SOCK_DGRAM: Supporta comunicazioni datagram, senza connessione (socket UDP) e con sequenze inaffidabili di messaggi
 - SOCK_RAW: per l'accesso diretto (raw socket) alle comunicazioni di rete (protocolli ed interfacce)
- Protocol - protocollo usato
 - Per noi ne esiste solo uno, UDP per SOCK_DGRAM, TCP per SOCK_STREAM
 - Possiamo impostarlo a 0

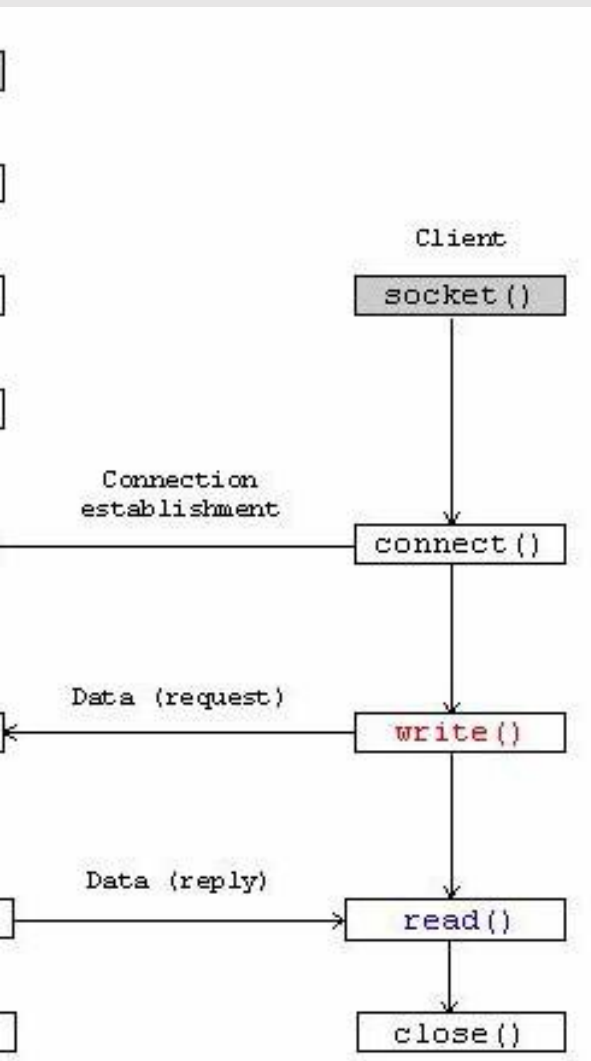


Anatomia **Server TCP** - *Psrv*

1. Psrv definisce la socket invocando `socket()`
 - Crea una *unnamed socket*, ovvero una struttura dati che rappresenta la socket, ma senza nome (indirizzo).
2. Psrv associa un nome alla socket invocando `bind()`
 - unnamed socket -> *named socket*
3. Psrv definisce la lunghezza della coda di ingresso – `listen`
 - ovvero il numero massimo gestibile di connessioni pending
4. Psrv si mette in ascolto sulla socket in attesa di una richiesta di connessione del client - `accept()`
 - `accept()` ritorna un fd usato per comunicare con il client
5. Psrv crea figlio che userà fd per comunicare con il client
6. Psrv si rimette in ascolto sulla socket con `accept()` per una nuova connessione

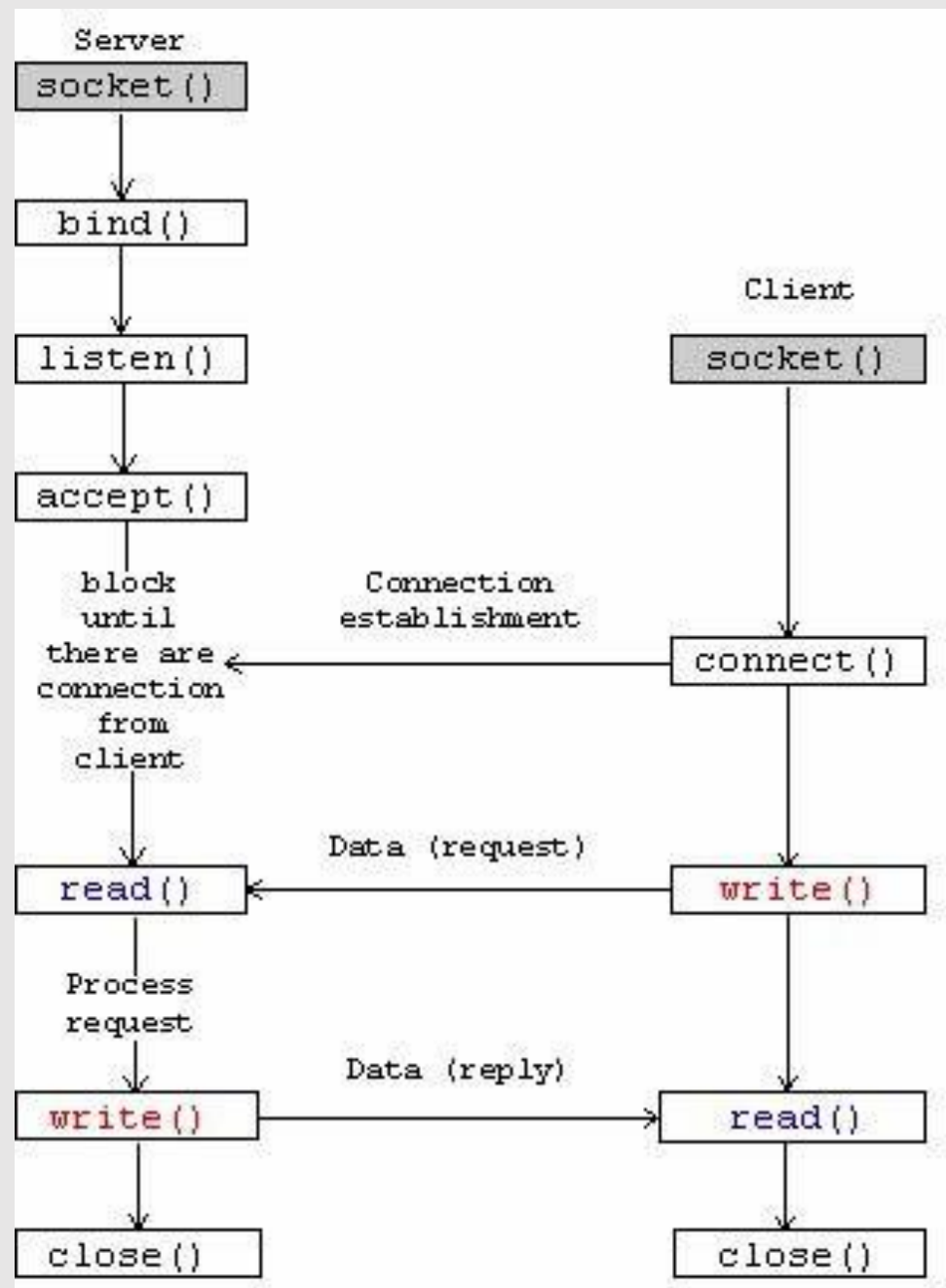


Anatomia *Client* TCP - *Pcli*



1. Pcli definisce il socket invocando `socket()`
 - Crea una *unnamed socket* (`csock`), ovvero una struttura dati che rappresenta la socket, ma senza nome (indirizzo).
2. Pcli imposta una struttura dati `sin` di tipo `struct sockaddr_in` in modo da scriverci le informazioni del server al quale si vuole connettere.
3. Pcli si connette al server invocando la syscall `connect()` alla quale passa il socket (`csock`) e l'indirizzo del server al quale connettersi.
 - La `connect` ritorna un descriptor del server (`ssd`) (o -1 se errore)
4. Pcli utilizza `ssd` per leggere (`read()`) e scrivere (`write()`) da/su server
5. Finita la necessità di comunicare con il server, Pcli chiude la connessione invocando `close(ssd)`

Interazione Client-Server



Struttura codice *Server*

```
int sd = socket(AF_INET, SOCK_STREAM, 0);  
bind(sd,...);  
listen(sd,MAX_QUEUED);  
while (1) {  
    int client_sd=accept(sd,...);  
    if (client_sd == -1){  
        perror("Errore accettando connessione dal client");  
        continue;  
    }  
    if (fork() == 0) { /*eseguito dal figlio */  
        /* read/write su client_sd */  
        close(client_sd);  
        exit(0);  
    }  
}
```

Struttura codice *Client*

```
main () {  
    int cfd = socket(AF_INET, SOCK_STREAM, 0);  
    /* set socaddr_in structure */  
    if (connect(cfd,...) != 0) {  
        perror("connessione non riuscita");  
    }  
    /* read(cfd,...) e write(cfd,...) da/verso server */  
    close(cfd);  
}
```

```
int socket(int domain, int type,  
int protocol);
```

```
#include <sys/types.h>  
#include <sys/socket.h>
```

- Ritorna -1 in caso di errore
- La scrittura/lettura su un socket chiuso genera un errore SIGPIPE

```
int sd;  
  
sd=socket(AF_UNIX, SOCK_STREAM, 0);  
if (sd == -1) {  
    perror("Creando Socket");  
    exit(-1);  
}
```

```
int bind(int sockfd, const struct sockaddr *addr,  
        socklen_t addrlen);
```

- `sockfd`: id unnamed socket
- `addr`: struttura di tipo `sockaddr` che contiene l'indirizzo IP/nome della socket
 - `struct socaddr_in` per `AF_INET` `AF_INET6`
 - `struct socaddr_un` per `AF_LOCAL`
- Solo `AF_INET` sockets possono fare binding su *IP_address:porta*
- `addrlen`: dimensione struttura `sockaddr`

struct sockaddr_in

```
struct sockaddr_in {
    sa_family_t sin_family;    /* address family: AF_INET */
    in_port_t sin_port;       /* port in network byte order */
    struct in_addr sin_addr;   /* internet address */
};

/* Internet address. */
struct in_addr {
    uint32_t s_addr; /* address in network byte order */
};
```

- `sin_port` e `sin_addr` sono binari in formato *network*.
- I numeri di porta del protocollo e gli indirizzi internet vanno quindi tradotti in formato *network*.
- È possibile assegnare un valore a `sin_addr` con le seguenti macro:
 - `INADDR_LOOPBACK` (127.0.0.1)
 - `INADDR_ANY` (0.0.0.0)



Funzioni per convertire indirizzi

- `uint32_t htonl(uint32_t hostlong);`
 - converte un unsigned int in formato network
- `int inet_aton(const char *cp, struct in_addr *inp);`
 - converte un indirizzo `cp="X.Y.Z.W"` in formato network (struct in_addr)
- `struct hostent *gethostbyname(const char *name)`
 - dato un nome (`mio.dominio.toplevel`) o un indirizzo `X.Y.Z.W` ritorna una struttura `hostent` che contiene l'indirizzo in formato network

```
int listen(int sockfd, int  
backlog);
```

- Marca la socket `sockfd` come *passive*, ovvero pronta a ricevere richieste mediante `accept()`
- `backlog` indica la lunghezza della coda di attesa
- Restituisce 0 in caso di successo e -1 in caso di errore

```
int accept(int sockfd,  
struct sockaddr *addr, socklen_t *addrlen);
```

- Usata per socket con connessione (non per SOCKET_DGRAM)
- Estrae la prima richiesta di connessione nella coda delle richieste in attesa della socket `sockfd`
- Crea una nuova socket con connessione e ritorna un nuovo file descriptor
 - nuova socket non è in ascolto
- La socket `sockfd` continua il suo lavoro

```
int connect(int sockfd,  
const struct sockaddr *addr,  
socklen_t addrlen);
```

- Invocata da un client per associare un indirizzo `addr` ad un unnamed socket `sockfd`
- `addrlen` è la dimensione di `addr` (ovvero `sockaddr`)
- Se va a buon fine ritorna 0 e quindi `sockfd` può essere utilizzato come file descriptor per leggere e scrivere su server.

Esempi ed esercizi

Debian/Ubuntu: per trovare il vostro indirizzo IP eseguite `ip addr` da terminale

- Vedi esempi
 - `echo_server.c`
 - `client.c server.c`
 - `client2.c server2.c`
- **Esercizio:** Modificare `client.c` e `server.c` in modo che la porta della connessione sia passata come parametro.
- **Esercizio:** Modificare il codice server in modo che, per ogni accept che va a buon fine, venga creato un processo che soddisfi la richiesta (mandando l'ora)



Domande?

